

Figure 5: The Scale Cube (Source: The Art of Scalability by Martin L. Abbott and Michael T. Fisher)

from a third party library that supports Java 8, but other components are not compatible and are still dependent on Java 6, it's difficult to do so.

Scaling the whole application when only a certain part needs scaling

Scaling requires the entire application to be scaled rather than just parts of it. This requires greater resources as shown in Figure 4.

It is possible to scale horizontally by running many instances behind the load balancer. Scaling can even be done on the basis of sorting the request origin. For example, if the request comes from the Asia region, use a predefined DNS server or database server; if the request is from the European region, use the other DNS or database server, etc. The art of scalability shows scaling on three axes. When using the monolithic approach, it is not possible to scale on functional decomposition. Figure 5 shows three different axes for scaling.

Apart from this, when a new person joins a team, he or she needs to understand all the layers and has to go through thousands of lines of code so as to become a productive team member. Any new innovative idea may not be accommodated or be implemented easily, due to partitioning of the team on the basis of technology, which leads to a communication gap. Changes require a lot of approvals, which may take a long time.

The Microservices approach

The Microservices architectural style gained momentum because it allowed building the application as suites of services. Each service does exactly one thing or implements one functionality; it is independent of other services, can be deployed independently and can even use a different stack of technology – one that is the most efficient for implementing a particular functionality.

Characteristics of Microservices architecture

According to Martin Flower, the common characteristics of Microservices architecture are:

Componentisation via services: Instead of building components, we can think in terms of services, which can run independently and will expose themselves as REST end points or some other mechanism.

Organised around business capabilities: We normally make teams on the basis of technology. Almost all large organisations have a UI team, a database team, an engineering team, etc — hence the three-layer architecture (Figure 1). Due to this, even a simple change needs to be communicated in a channel, and may take a long time for approval.

In the Microservices approach, the teams are cross-functional and are built according to business capabilities, such as the UI, database, project management, code, testing, etc. Each team is responsible for one service, which it builds and runs. The team is solely responsible for that service. The team size ideally follows the 'two pizza rule' that Amazon CEO Jeff Bezos came up with — the size of a team should not be larger than what two pizzas can feed.

Products not projects: In the monolithic approach, a project is built by a team, goes for testing and then moves to the maintenance phase. This is a typical project model.

In Microservices, the team is built around the service and the product it builds; the team owns it for its full life time. The Amazon concept is: "You build and you run it."

Smart endpoints and dumb pipes: In the SOA based approach, there is the concept of an enterprise bus which is like the brain of the system. It does everything — from message routing and transformation to applying business rules, and so on.

In Microservices, on the other hand, the bus is dumb, as the services need to be decoupled and cohesive. It uses HTTP for communication, and may use REST or some other way to

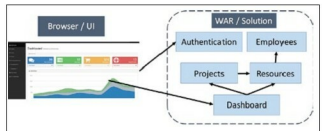


Figure 6: Conceptual monolithic application (Source: 'Predicting the future of Microservices and SPA' by Keyhole Software LLC)

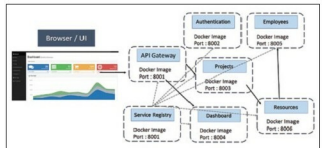


Figure 7: Monolithic application decomposed in Microservices (Source: 'Predicting the future of Microservices and SPA' by Keyhole Software LLC)